

# Reviewer Pack — Irreducible Component Count Inversely Proportional to $\text{ACC}^0$ C...

Entry f5574a6cc9ec · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-12 23:21:39 UTC

## Irreducible Component Count Inversely Proportional to $\text{ACC}^0$ Circuit Size for Sipser-like Functions

**Entry ID:** f5574a6cc9ec **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-12 23:21:39 UTC

### 1. Conjecture

**Field A** (mathematical branch): Algebraic Geometry over Finite Fields **Field B** (complexity object):  $\text{ACC}^0$  Circuit Size for Explicit Functions

#### Statement:

For a random SAT instance with  $n$  variables, the number of irreducible components of its defining polynomial over  $\text{GF}(2)$  is at least  $\Omega(2^{\{n/2\}})$  if the function cannot be computed by an  $\text{ACC}^0$  circuit of size  $o(2^{\{n/2\}})$ .

#### Rationale (proposer's reasoning):

Algebraic geometry provides a framework to analyze the structure of Boolean functions via their polynomial representations. The irreducible components of the defining polynomial encode the function's geometric complexity, which may correlate with circuit size barriers in  $\text{ACC}^0$ .

**Taxonomy category:** ACC\_SIPSER (status at proposal time: alive)

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix):** 555e3d9af07a2c16

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

#### Acceptance criterion:

default:  $\geq 80\%$  seeds must support, no counterexample

### 3. Barrier filter (F1)

**Final:** PASS

| Barrier        | Verdict | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|----------------|---------|------------|------------------|------------------|
| RELATIVIZATION | SAFE    | 0.00       | UNCERTAIN        | UNCERTAIN        |
| NATURAL_PROOFS | SAFE    | 0.00       | UNCERTAIN        | UNCERTAIN        |

| Barrier       | Verdict | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|---------------|---------|------------|------------------|------------------|
| ALGEBRIZATION | SAFE    | 0.00       | UNCERTAIN        | UNCERTAIN        |
| KARP_LIPTON   | SAFE    | 0.95       | UNCERTAIN        | SAFE             |

## 4. Novelty audit

**Verdict:** NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

## 5. Empirical test harness

**Multi-seed protocol:** 5 seeds (11, 23, 37, 53, 71)

**Sandbox:** isolated subprocess, stdlib-only,  $\leq 90$ s wall time per cycle **Execution:** rc=1, elapsed=0.2s

### 5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def is_prime(n):
        if n <= 1:
            return False
        for i in range(2, int(math.sqrt(n)) + 1):
            if n % i == 0:
                return False
        return True

    def generate_primes(limit):
        primes = []
        for num in range(2, limit):
            if is_prime(num):
                primes.append(num)
        return primes

    def generate_random_3sat_instance(n):
        clauses = []
        for _ in range(3 * n):
            clause = [random.choice([-1, 1]) * random.randint(1, n) for _ in range(3)]
            if len(set(clause)) == 3:
                clauses.append(clause)
        return clauses

    def gf2_polynomial_from_3sat(clauses):
        n = max(abs(var) for clause in clauses for var in clause)
        poly = [0] * (1 << n)
        for clause in clauses:
            product = 1
            for var in clause:
                if var > 0:
                    product ^= 1 << (var - 1)
                else:
                    product ^= 1 << (-var - 1)
```

```

        poly[product] += 1
    return poly

def factorize_poly(poly):
    n = len(poly)
    factors = []
    for i in range(1, n):
        if poly[i] != 0 and all(poly[j] % poly[i] == 0 for j in range(i + 1, n)):
            factors.append([i])
            break
    return factors

def count_irreducible_components(poly):
    return len(factorize_poly(poly))

def acc0_circuit_size_estimate(n):
    return 2 ** (n // 2)

n = random.choice([5, 10, 15, 20, 30, 40])
instance = generate_random_3sat_instance(n)
poly = gf2_polynomial_from_3sat(instance)
irreducible_components_count = count_irreducible_components(poly)
acc0_size_estimate = acc0_circuit_size_estimate(n)

return {
    "metric_name": "irreducible_components_count",
    "metric_value": irreducible_components_count,
    "instances_tested": 1,
    "conjecture_holds": irreducible_components_count >= acc0_size_estimate,
    "counterexample": "" if irreducible_components_count >= acc0_size_estimate else f"n={n}, p={p}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or generate_primes(100)[:30]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value:.2f} std={std_value:.2f} support_fraction={support_fraction:.2f}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={result['counterexample']} first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

## 6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

## 7. Test stdout (last 2KB)

[illegible]

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

**Reasoning:**

Test crashed before producing conclusive data; pre-registered support condition cannot be verified | next: Run test with increased time/memory limits and verify output format

## 11. Audit log (LLM calls)

**Total LLM calls: 7**

| # | Phase           | Provider      | Model            | Tokens in | out | Latency (ms) |
|---|-----------------|---------------|------------------|-----------|-----|--------------|
| 1 | propose         | ollama_remote | qwen3:8b         | 0         | 0   | 100740       |
| 2 | preregistration | ollama_remote | qwen3:8b         | 0         | 0   | 24015        |
| 3 | novelty         | ollama_remote | qwen3:8b         | 0         | 0   | 20743        |
| 4 | novelty         | ollama_remote | qwen3:8b         | 0         | 0   | 16400        |
| 5 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 20391        |
| 6 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 9659         |
| 7 | judge           | ollama_remote | qwen3:8b         | 0         | 0   | 19681        |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 211629 ms total latency. Provider mix: {'ollama remote': 7}

(full prompt+response transcripts available in `research/audit/f5574a6cc9ec.jsonl`)

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/f5574a6cc9ec.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** `research/pvsnp sandbox/test *.py` (per-cycle)

**Replay tarball:** `research/replay/f5574a6cc9ec.tar.gz` (if generated)